



SuperEarn: Crosschain Security Review

Cantina Managed review by:

Ladboy233, Security Researcher

Drastic Watermelon, Associate Security Researcher

May 21, 2026

Contents

1	Introduction	3
1.1	About Cantina	3
1.2	Disclaimer	3
1.3	Risk assessment	3
1.3.1	Severity Classification	3
2	Security Review Summary	4
2.1	Scope	4
2.2	Cantina Managed Team Statement	5
3	Findings	6
3.1	Critical Risk	6
3.1.1	<code>RemoteVault.totalAssets</code> adds amounts of different denominations	6
3.2	High Risk	6
3.2.1	Incorrect asset to share conversion in <code>RemoteVault._depositToYearn</code>	6
3.2.2	<code>RemoteVault</code> deposits and withdrawals can be frozen indefinitely	6
3.2.3	<code>retrieveDebt</code> should have access control	9
3.3	Medium Risk	10
3.3.1	Funds held by <code>SuperEarnRouter</code> can be taken by anyone	10
3.3.2	Calling <code>CrosschainAdapter.setAccountant</code> will reset all crosschain asset accounting	11
3.3.3	<code>getSupplyCap()</code> logic in <code>StrategyMorphorV1Vault</code> can block <code>adjustPosition</code>	11
3.3.4	Hardcoded Incorrect min sqrt ratio breaks boundary price math in <code>UniversalSwapRouter.sol</code>	12
3.3.5	<code>OriginVault.emergencyFulfillRedeem</code> will always revert	12
3.4	Low Risk	12
3.4.1	<code>RemoteVault.setYearnVault</code> may be easily DoS'd	12
3.4.2	<code>OriginVault</code> redemption requests can get stuck with no way to cancel	13
3.4.3	<code>OraklAssetPriceConverter</code> assumes 8 decimal precision for all feeds	13
3.4.4	Incorrect access control used in <code>RemoteVault</code> emergency methods	14
3.4.5	<code>OriginVault._convertAssetToUsdt</code> assumes no difference in decimal precision between assets	14
3.4.6	<code>ERC7540Vault.sol#tendTrigger</code> does not check if the vault is in <code>emergencyExit</code> state	14
3.4.7	Upper bound and lower bound oracle price assumption can impact protocol solvency	14
3.4.8	<code>OriginVault.emergencyWithdrawFromAgent</code> can trigger double-counting errors	15
3.4.9	<code>batchFulfillRedemptions</code> contains unbounded loop in <code>OriginVault.sol</code>	15
3.4.10	<code>estimatedTotalAssets</code> should use <code>previewRedeem</code> and instead of <code>previewMint</code> to estimate total asset	15
3.4.11	The system heavily relies on unbounded loops	15
3.4.12	Calls to <code>CooldownVault.recover</code> dilute value for all share holders	16
3.4.13	Two-step execution pattern in <code>BaseCooldownStrategy</code> lacks access control separation and time delay enforcement	16
3.5	Gas Optimization	16
3.5.1	Execute allowance check early	16
3.5.2	Mathematical calculation can be simplified	17
3.5.3	<code>SuperEarnAccessControl._isContract</code> is used only within permissioned methods	17
3.5.4	Avoid try/catch-ing external calls to revert in the catch clause	17
3.5.5	Unnecessary <code>Math.min</code> call	17
3.5.6	Token transfers in <code>SuperEarnMessageAgent</code> can be simplified into one	18
3.6	Informational	18
3.6.1	Incorrect comment	18
3.6.2	<code>UniversalSwapRouter</code> uses two libraries for safe <code>ERC20</code> interactions	18
3.6.3	<code>CrosschainAdapter.sendAssets</code> will incorrectly account for messages sent to un-registered networks	18
3.6.4	Incorrect parameter in external call within <code>CrosschainAdapter.sendAssets</code>	19
3.6.5	Empty method implements access control	19

3.6.6	<code>SuperEarnAccessControl</code> modifiers should use the contract's getters	19
3.6.7	Variable assignment can be simplified	19
3.6.8	Incorrect parameter name in interface's event	20
3.6.9	Initial infinite approval from <code>OriginVault</code> to <code>SuperEarnMessageAgent</code> is overwritten	20
3.6.10	Unreachable code	20
3.6.11	<code>HealthCheck.setGovernance</code> is missing access control	20
3.6.12	Unnecessary ternary operator	21
3.6.13	Bridge accounting assumes single asset type and fails to track per-token balances . .	21
3.6.14	<code>CooldownVault</code> isn't ERC4626 compliant	21

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

SuperEarn delivers stablecoin yields in a secure, smart, simple way.

From Oct 29th to Nov 19th the Cantina team conducted a review of [superearn-core](#) on commit hash [78074879](#). The team identified a total of **42** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	1	1	0
High Risk	3	3	0
Medium Risk	5	5	0
Low Risk	13	11	2
Gas Optimizations	6	6	0
Informational	14	13	1
Total	42	39	3

2.1 Scope

The security review had the following components in scope for [superearn-core](#) on commit hash [78074879](#):

```
src/superearn
├── core
│   ├── BaseCooldownStrategy.sol
│   ├── CooldownVault.sol
│   ├── CooldownVaultFactory.sol
│   ├── HealthCheck.sol
│   ├── StrategyERC7540Vault.sol
│   ├── StrategyMorphoV1Vault.sol
│   └── StrategyMorphoV1Vault.sol.bak
├── interface
│   ├── IBaseFeeOracle.sol
│   ├── ICooldownVault.sol
│   ├── IRegistry.sol
│   ├── IStrategy.sol
│   ├── IStrategyCooldownAware.sol
│   ├── ISuperEarnRouter.sol
│   └── IVault.sol
├── periphery
│   └── SuperEarnRouter.sol
├── v2
│   ├── base
│   │   └── SuperEarnAccessControl.sol
│   ├── core
│   │   ├── crosschain
│   │   │   ├── BridgeAccountant.sol
│   │   │   ├── BridgeQueue.sol
│   │   │   ├── CrosschainAdapter.sol
│   │   │   └── SuperEarnMessageAgent.sol
│   │   └── vaults
│   │       ├── OriginVault.sol
│   │       ├── RemoteVault.sol
│   │       └── SuperearnERC7540.sol
│   └── interfaces
│       ├── IBridgeAccountant.sol
│       ├── ICrosschainAdapter.sol
│       ├── ICrosschainVault.sol
│       └── IOriginVault.sol
```

```

├── IRemoteVault.sol
├── IRunespearAgent.sol
├── IRunespearReceiver.sol
├── ISwapQuoter.sol
├── libraries
│   ├── AssetPriceConverter.sol
│   ├── OraklAssetPriceConverter.sol
│   └── VaultStateHelper.sol
├── messaging
│   ├── runespear
│   │   ├── RunespearLib.sol
│   │   ├── RunespearProtocol.sol
│   │   ├── RunespearReceiver.sol
│   │   ├── RunespearSender.sol
│   │   └── RunespearTransceiver.sol
│   └── SuperEarnV2Protocol.sol
└── periphery
    ├── SwapQuoter.sol
    └── UniversalSwapRouter.sol

```

2.2 Cantina Managed Team Statement

During the engagement, the Cantina Managed team identified findings across multiple severity levels and observed areas where the codebase's complexity made security-critical logic harder to reason about. Throughout the review, the protocol team was highly responsive, addressing issues promptly and removing outdated code as it was surfaced.

Following the initial review, the protocol team has:

- Remediated all Critical, High, and Medium severity findings, with the fixes subsequently verified by Cantina.
- Simplified the codebase and expanded the end-to-end integration test suite, particularly around the cross-chain components that represent the most security-sensitive areas of the system.
- Completed an additional full security review of the updated codebase.
- Planned a public bug bounty program to provide ongoing assurance after deployment.

These remediation steps materially improve the security posture of the protocol relative to the state observed during the initial review. As with any system of this scope, we encourage the team to maintain strong testing discipline and continued external scrutiny, including the upcoming bug bounty, as the protocol evolves in production.

3 Findings

3.1 Critical Risk

3.1.1 `RemoteVault.totalAssets` adds amounts of different denominations

Severity: Critical Risk

Context: `RemoteVault.sol#L336-L343`

Description: `RemoteVault.totalAssets` should return the amount of underlying asset held by the vault. The `RemoteVault.assetsInTransitToOrigin` returns an amount of `USDT` instead, which is the denomination bridged back and forth between `OriginVault` and `RemoteVault`. As a result, the value returned by `RemoteVault.totalAssets` is incorrect, causing errors in the asset to share conversion within the core `ERC4626` methods.

Recommendation: Within `RemoteVault.totalAssets`, convert the result of `assetsInTransitToOrigin()` to the vault's asset.

SuperEarn: Fixed in commit `6d384bb6`.

Cantina Managed: Fix verified.

3.2 High Risk

3.2.1 Incorrect asset to share conversion in `RemoteVault._depositToYearn`

Severity: High Risk

Context: `RemoteVault.sol#L628`

Description: `RemoteVault._depositToYearn` is triggered by a call to the permissioned `RemoteVault.depositToYearn` method. Such method is used to make a `RemoteVault` instance deposit a specified amount of assets into the integrated Yearn vault.

The method incorrectly converts assets to shares without taking into consideration their relative exchange rates: in particular, it both ignores the conversion rates between assets and `CooldownVault` shares, and between the latter and Yearn vault shares.

Because of the above, depending on the exchange rates, the obtained `minSharesOut` may be over- or under-estimated by a wide margin, resulting in either:

- A larger slippage margin than desired, which may result in greater financial losses than acceptable.
- A smaller slippage margin than desired, which will trigger unexpected transaction failures.

Recommendation: In order to correctly convert an amount of assets to Yearn vault shares, the method should:

1. Convert assets to `CooldownVault` shares → use `CooldownVault.convertToShares`.
2. Convert `CooldownVault` shares to Yearn vault shares → use `Vault.pricePerShare` to calculate the corresponding amount of Yearn vault shares.

In order to maintain consistency with the `RemoteVault._withdrawFromYearn` method, the team could implement a `SuperEarnRouter.previewDeposit` which encapsulates the logic shown above and may be used within `RemoteVault._depositToYearn`.

SuperEarn: Fixed in commit `08ec4186`.

Cantina Managed: Fix verified.

3.2.2 `RemoteVault` deposits and withdrawals can be frozen indefinitely

Severity: High Risk

Context: (No context files were provided by the reviewer)

Summary: RemoteVault inherits OpenZeppelin's ERC4626 implementation, overriding most of its functionality. Crucially, it overrides the ERC4626.totalAssets method to take into consideration different assets distributed to different parts of the system and not held by the RemoteVault specifically.

Part of such assets are account within the RemoteVault._calculatePendingCooldownAssets method, which is meant to account for any CooldownVault redemption request held by the contract.

Finding Description: The RemoteVault._calculatePendingCooldownAssets method calculates the amount of assets it has in pending redemption request in very inefficient manner:

1. It fetches all unclaimed redemption requests by calling cooldownVault's getUnclaimedRedeemRequestIds() method.
2. It loops through all of the received redemption requests, filtering for those unclaimed and which have receiver set to RemoteVault, accumulating the amount of assets.

This unbounded loop enables any account to leverage a bloat in the value returned by cooldownVault.getUnclaimedRedeemRequestIds() to make RemoteVault._calculatePendingCooldownAssets's gas consumption grow without a limit.

Specifically, a malicious actor is able to force the method to consume more than 60 million gas, making it impossible for anyone to call the method once the contract is deployed to Ethereum. The inability to correctly call RemoteVault.totalAssets implies the inability to call every core ERC4626 methods (mint, deposit, redeem, withdraw) as they all rely on internal calls to RemoteVault.totalAssets to determine the conversion price between assets and shares.

Analyzing further, the inability for RemoteVault.deposit, RemoteVault.withdraw and similar methods to be called implies a full shutdown of core protocol functionalities.

Note 1: Notice that, although anyone may claim a redemption request with a reasonable slippage control, and thus remove a given redemption request from the set of unclaimed ones, an attacker can prevent this by setting the redemption request's receiver to be an account which has been blacklisted on the USDC or USDT contract, which will make any transfer of tokens to such account fail, ultimately causing the redemption claim to fail as well.

Note 2: Notice that, following Ethereum's Fusaka upgrade in which EIP-7825 is to be deployed, a cap on any transaction's gas consumption of 16,777,216 will be applied. This implies that the analysis executed for this issue will have to be adjusted to account for such limit: roughly speaking, a reduction by 1/4th of the needed pending redemption requests is expected to suffice for the issue to manifest.

Proof of Concept: In order to verify the finding, we've created the following PoC in which we create a large number of redemption requests and measure the amount of gas required for the highlighted method to be executed, asserting that it is larger than Ethereum's block gas limit. Within our research, we've found that 14000 redemption requests will trigger a large enough gas consumption.

In order to execute the test case, use `FOUNDRY_PROFILE=MAINNET forge t --mt test_RedeemVaultDoS`.

```
pragma solidity >=0.8.29;

import "forge-std/src/Test.sol";
import "@superearn/core/CooldownVault.sol";
import "@superearn/core/CooldownVaultFactory.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@superearn/interface/ICooldownVault.sol";

// Simple mock ERC20 for testing
contract MockToken is ERC20 {
    uint8 private _decimals;

    constructor(string memory name, string memory symbol, uint8 decimals_) ERC20(name,
    ↪ symbol) {
        _decimals = decimals_;
    }

    function mint(address to, uint256 amount) external {
```



```

        _mint(to, amount);
    }

    function decimals() public view override returns (uint8) {
        return _decimals;
    }
}

contract PoC_GasDoS_CooldownRedemptions is Test {
    CooldownVault public cooldownVault;
    CooldownVaultFactory public factory;
    MockToken public usdc;

    address public governance = makeAddr("governance");
    address public attacker = makeAddr("attacker");
    address public remoteVault = makeAddr("remoteVault");

    uint256 constant COOLDOWN_PERIOD = 7 days;
    uint256 constant INITIAL_BALANCE = 1_000_000e6;
    uint256 constant ATTACKER_DEPOSIT = 100e6;

    // Ethereum gas limit is 60M per block
    uint256 constant ETH_BLOCK_GAS_LIMIT = 60_000_000;

    function setUp() public {
        usdc = new MockToken("USDC", "USDC", 6);

        factory = new CooldownVaultFactory();

        usdc.mint(address(this), 1e4);
        usdc.approve(address(factory), 1e4);

        cooldownVault = factory.createCooldownVault(
            IERC20(address(usdc)),
            "Cooldown USDC",
            "cUSDC",
            COOLDOWN_PERIOD,
            governance
        );

        usdc.mint(attacker, ATTACKER_DEPOSIT);
    }

    // AUDIT copied here to simulate internal call jump instead of external call
    function calculatePendingCooldownAssets(address receiver) public view returns
    ↪ (uint256) {
        // Get all unclaimed redemption request IDs from CooldownVault
        uint256[] memory unclaimedIds = cooldownVault.getUnclaimedRedeemRequestIds();

        uint256 totalPending = 0;
        for (uint256 i = 0; i < unclaimedIds.length; i++) {
            uint256 requestId = unclaimedIds[i];

            // Query individual redemption request details
            (
                address requestReceiver,
                uint256 assets,
                // cooldownRequestedTime
                // cooldownPeriod
                bool claimed
            ) = cooldownVault.redeemRequests(requestId);

            // Only count requests where WE are the receiver and not yet claimed
            if (requestReceiver == receiver && !claimed) {
                totalPending += assets;
            }
        }
    }
}

```

```

    }

    return totalPending;
}

function test_RedeemVaultDoS() public {
    // attacker gets cooldown vault shares
    vm.startPrank(attacker);
    usdc.approve(address(cooldownVault), ATTACKER_DEPOSIT);
    cooldownVault.deposit(ATTACKER_DEPOSIT, attacker);
    vm.stopPrank();

    // create redemption requests
    uint256 toCreate = 14_000;
    vm.startPrank(attacker);
    for (uint256 i; i < toCreate; i++) {
        cooldownVault.redeem(1, attacker, attacker);
    }
    vm.stopPrank();

    // measure gas cost
    uint256 gasStart = gasleft();
    calculatePendingCooldownAssets(remoteVault);
    uint256 gasUsed = gasStart - gasleft();

    assertGe(gasUsed, ETH_BLOCK_GAS_LIMIT);
}
}

```

Recommendation: The system should completely avoid unbounded loops, especially over a end user controlled data set like the unclaimed redemption requests. The protocol should consider refactoring the system to allow spot queries (e.g. to a mapping) to retrieve the necessary data, in this case the total amount of assets sitting in unclaimed redemption requests attributed to a given account.

SuperEarn: Fixed in commit [4b4b43e1](#).

Cantina Managed: Fix verified.

3.2.3 `retrieveDebt` should have access control

Severity: High Risk

Context: `CooldownVault.sol#L499-L523`

Description: After the strategy mint shares to incurs debt, after the timelock, anyone can trigger `retrieveDebt` to remove the `totalDebt`. Even the strategy underpays the debt, the full debt is removed: `request.debtAssets`.

Because the function has no access control, a malicious actor can intentionally select a time slot when the `Cooldown strategy` is short of asset to trigger the debt repayment and socialize the loss to the cooldown vault share holder.

```

if (alreadyClaimed) {
    if (isDoneActually) {
        // don't need to claim again
        claimedAmount = Math.min(externalUnderlyingToken.balanceOf(address(this)),
            ↳ redeemUnderlyingAmount);
    } else {
        revert DebtClaimedNotYet();
    }
}

```

Recommendation: Add access control and only allows operator to trigger `retrieveDebt`.

SuperEarn: Fixed in commit [7c5da766](#).

Cantina Managed: Fix verified.

3.3 Medium Risk

3.3.1 Funds held by SuperEarnRouter can be taken by anyone

Severity: Medium Risk

Context: SuperEarnRouter.sol#L56-L94

Description: SuperEarnRouter may be used by end users to batch deposits from assets, into CooldownVault shares and then into a Yearn vault shares. SuperEarnRouter.deposit must be provided with the target Yearn vault which represents the end target of the user's funds. Because the contract doesn't implement a whitelist to ensure that the method is provided only with protocol-deployed vaults, it is possible for anyone to skim any funds that may be held by the SuperEarnRouter.

Proof of Concept: Create a tests/SuperEarnRouter.t.sol test file, insert the following content and execute with `forge t --mt test_deposit_stealFundsOnRouter`.

```
pragma solidity >=0.8.29;

import { Test } from "forge-std/src/Test.sol";
import { SuperEarnRouter } from "../src/superearn/periphery/SuperEarnRouter.sol";

import "../src/mock/MockToken.sol";

contract SuperEarnRouterTest is Test {
    SuperEarnRouter internal superEarnRouter;

    address registry;
    address attacker;

    MockToken token;

    function setUp() public virtual {
        registry = makeAddr("registry"); // used only in SuperEarnRouter.endorsedVault
        attacker = makeAddr("attacker");

        superEarnRouter = new SuperEarnRouter(registry);
        token = new MockToken("mock", "mock", 18);
    }

    function test_deposit_stealFundsOnRouter() external {
        // Attacker has 1 wei of tokens
        uint256 attackerInitialBalance = 1;
        token.mint(attacker, 1);

        // Some tokens are on router
        uint256 routerBalance = 100e18;
        token.mint(address(superEarnRouter), routerBalance);

        // Attacker deploys his contracts
        vm.startPrank(attacker);
        CooldownVault voidVault = new CooldownVault(address(token));
        Vault yVault = new Vault(address(voidVault));

        // Attacker skims assets from router
        token.approve(address(superEarnRouter), 1);
        superEarnRouter.deposit(address(yVault), 1, attacker, 0);

        // Assert attacker received funds
        assertEq(token.balanceOf(attacker), routerBalance + attackerInitialBalance,
            ↪ "attack failed");
        assertEq(token.balanceOf(address(superEarnRouter)), 0, "router still has
            ↪ tokens");
    }
}
```

```

contract Vault {
    address public cooldownVault;
    constructor(address _cooldownVault) {
        cooldownVault = _cooldownVault;
    }
    function token() external view returns(address) {return cooldownVault;}
    function deposit(uint256,address) external pure returns(uint256) {return 1;}
}

contract CooldownVault {
    address public asset;
    constructor(address _asset) {
        asset = _asset;
    }
    function deposit(uint256,address) external view returns(uint256) {return 1;}
    function approve(address,uint256) external {}
}

```

Recommendation: Implement a vault whitelist and check the highlighted method's `yVault` parameter has been whitelisted.

SuperEarn: Fixed in PR 7.

Cantina Managed: Fix verified.

3.3.2 Calling `CrosschainAdapter.setAccountant` will reset all crosschain asset accounting

Severity: Medium Risk

Context: `CrosschainAdapter.sol#L576-L581`

Description: `CrosschainAdapter.setAccountant` may be used to change the `CrosschainAdapter.accountant` storage variable, which is fetched by `SuperEarnMessageAgent.getTruePeerAssets` and `SuperEarnMessageAgent.getAssetsInTransit`. The highlighted `SuperEarnMessageAgent` methods are both used within `OriginVault.totalAssets`. As a consequence, a change to `CrosschainAdapter.accountant` will result in a complete reset of all crosschain assets accounting, unless special care is taken to recreate the current accounting state within the new accountant before it is set with `CrosschainAdapter.setAccountant`.

Recommendation: Consider removing the highlighted method and making the `CrosschainAdapter.accountant` value an immutable.

SuperEarn: Fixed in commit e154a608.

Cantina Managed: Fix verified.

3.3.3 `getSupplyCap()` logic in `StrategyMorphoV1Vault` can block `adjustPosition`

Severity: Medium Risk

Context: `StrategyMorphoV1Vault.sol#L169-L174`

Description: `ERC4626 ·maxDeposit(receiver)` returns the amount of assets that can be deposited right now (the remaining capacity or delta), not the global total asset ceiling. If `totalUnderlying` is greater than the remaining space (`supplyAssetsCap`), `Math.min(supplyAssetsCap, totalUnderlying)` is `supplyAssetsCap`.

```
availableAssets = (supplyAssetsCap - Math.min(supplyAssetsCap, totalUnderlying));
```

And the `availableAssets` is zero. The strategy assumes `maxDeposit` returns the Total Cap, but it returns available cap. The strategy will believe it has 0 available space to deposit, preventing `adjustPosition` from ever investing funds.

```
availableAssets = Math.min(availableAssets, MAX_SUPPLY_THRESHOLD);
```

Recommendation:

```
function getSupplyCap() public view virtual override returns (uint256 supplyAssetsCap,  
    ↪ uint256 availableAssets) {  
    supplyAssetsCap = metaMorpho.maxDeposit(address(this));  
    availableAssets = Math.min(supplyAssetsCap, MAX_SUPPLY_THRESHOLD);  
}
```

SuperEarn: Fixed in commit 5274303a.

Cantina Managed: Fix verified.

3.3.4 Hardcoded Incorrect min sqrt ratio breaks boundary price math in UniversalSwapRouter.sol

Severity: Medium Risk

Context: UniversalSwapRouter.sol#L360-L377

Description: The Uniswap v3/v4 TickMath docs tie those constants back to getSqrtRatioAtTick(MIN_TICK/MAX_TICK):

```
// v4 sqrt price bounds (Q64.96), same as Uniswap v3/v4 TickMath  
uint160 constant MIN_SQRT_RATIO = 4295128739;  
uint160 constant MAX_SQRT_RATIO =  
    1461446703485210103287273052203988822378723970342;
```

Currently the code hardcoded the MIN_SQRT_RATIO to incorrect value that does not comply the Uniswap doc. The incorrect hard-coded MIN_SQRT_RATIO can cause price-range math to break at extreme ticks, leading to mispriced swaps or reverted operations near pool boundaries.

Recommendation:

```
uint160 internal constant MIN_SQRT_PRICE = 4295128739;
```

SuperEarn: Fixed in commit 24703bb3.

Cantina Managed: Fix verified.

3.3.5 OriginVault.emergencyFulfillRedeem will always revert

Severity: Medium Risk

Context: OriginVault.sol#L792-L810

Description: OriginVault.emergencyFulfillRedeem may be used by an authorized party to fulfill an individual redemption request manually. Based on the distribution of assets between OriginVault and RemoteVault, the highlighted method may revert with a SlippageExceeded error.

While assetsNeeded is calculated by taking a portion of asset.balanceOf(address(this)), assetsToLock goes through OriginVault.totalAssets which also takes into consideration remote assets. Because the slippage check at OriginVault#L810 is executed assetsNeeded and a portion of assetsToLock, in the case in which RemoteVault holds more assets than OriginVault, the mentioned check will fail consistently unless special action is taken.

Recommendation: Remove the slippage check: the highlighted method is intended to be used in emergency scenarios which may not need a slippage check entirely. In alternative, implement the slippage check as a caller-provided parameter which the outgoing amount of assets must be greater of.

SuperEarn: Fixed in PR 6.

Cantina Managed: Fix verified.

3.4 Low Risk

3.4.1 RemoteVault.setYearnVault may be easily DoS'd

Severity: Low Risk

Context: RemoteVault.sol#L188-L189

Description: RemoteVault.setYearnVault may be used by a privileged account to change the RemoteVault.yearnVault storage variable. In the case in which yearnVault != address(0) and a new vault is being assigned to such variable, the highlighted method requires that yearnVault.balanceOf(remoteVault) == 0. This implies that any account may transfer a minimal amount of shares to the contract to block the migration from being successfully executed.

Recommendation: Consider whether such method is indeed useful to the RemoteVault, given that its accounting is tightly tied to the utilized Yearn vault. If so, the method could transfer any shares it holds to msg.sender.

SuperEarn: Fixed in commit 859b2f11.

Cantina Managed: Fix verified.

3.4.2 OriginVault redemption requests can get stuck with no way to cancel

Severity: Low Risk

Context: OriginVault.sol#L521

Description: The OriginVault.processRedemptionQueue function allows authorized accounts to fulfill pending redemption requests. The function accepts an asset amount limit and processes requests sequentially in FIFO (first-in, first-out) order until encountering a request that cannot be fully satisfied with the remaining allocation. The protocol lacks a mechanism to cancel redemption requests, creating a blocking condition where smaller redemption requests become indefinitely stuck behind larger unfulfillable requests in the queue.

Recommendation: Implement one of the following solutions:

1. Cancellation mechanism: Add a cancelRedemptionRequest function allowing users to remove their requests from the queue, enabling them to resubmit with adjusted amounts if needed.
2. Skip logic: Modify processRedemptionQueue to skip over requests that exceed the remaining asset limit and continue processing subsequent fulfillable requests.

SuperEarn: Acknowledged. If we allow cancellations, a redemption request would trigger a withdrawal from the strategy. However, if the external source integrated with the strategy does not support canceling redemptions, the Yearn shares would remain outstanding while the actual operating capital decreases. This would result in the funds not earning yield during the external source's cooldown period. Since this impact becomes more severe as the canceled amount grows — ultimately reducing the protocol's overall revenue — we decided not to support cancellation at the team level.

Cantina Managed: Acknowledged.

3.4.3 OraklAssetPriceConverter assumes 8 decimal precision for all feeds

Severity: Low Risk

Context: OraklAssetPriceConverter.sol#L9-L10

Description: OraklAssetPriceConverter.MIN_STABLECOIN_PRICE and OraklAssetPriceConverter.MAX_STABLECOIN_PRICE are defined with 8 units of decimal precision, which isn't guaranteed to match the decimals used by all price feeds.

Using a price feed with a different amount of decimals of fixed precision results in all calls to OraklAssetPriceConverter.getLatestData to fail with a PriceOutOfRange error.

Recommendation: In order to make the contract more flexible, these constants should be defined.

SuperEarn: Fixed in commit a26e63a2.

Cantina Managed: Fix verified.

3.4.4 Incorrect access control used in RemoteVault emergency methods

Severity: Low Risk

Context: RemoteVault.sol#L1026-L1036, RemoteVault.sol#L1044-L1056

Description: emergencyCooldownVaultRedeem and emergencyCooldownVaultClaim use the onlyManagers modifier instead of the onlyGovernance one.

Recommendation: Apply the onlyGovernance modifier to the highlighted methods and remove the onlyManagers modifier.

SuperEarn: Fixed in commit f47835bc.

Cantina Managed: Fix verified.

3.4.5 OriginVault._convertAssetToUsdt assumes no difference in decimal precision between assets

Severity: Low Risk

Context: OriginVault.sol#L345-L352, OriginVault.sol#L349

Description: The highlighted method assumes that assetAmount is expressed in the same decimals as USDT on the Kaia blockchain.

Recommendation: Modify the method take into account any possible difference in decimal precision between the two assets.

SuperEarn: Fixed in commit aa503a9a.

Cantina Managed: Fix verified.

3.4.6 ERC7540Vault.sol#tendTrigger does not check if the vault is in emergencyExit state

Severity: Low Risk

Context: StrategyERC7540Vault.sol#L729-L737

Description: tendTrigger in Morpho vault return false when emergencyExit is activated. The same check is missing in ERC7540Vault.sol, the impact is that even when the contract is in emergency mode, the offchain bot that relies on ERC7540Vault.sol still trigger harvest.

Recommendation: Add the check in ERC7540Vault.sol#tendTrigger function as well.

```
if (emergencyExit) return false;
```

SuperEarn: Fixed in commit 5934ae87.

Cantina Managed: Fix verified.

3.4.7 Upper bound and lower bound oracle price assumption can impact protocol solvency

Severity: Low Risk

Context: OraklAssetPriceConverter.sol#L9-L10

Description: The code hardcode MIN_STABLECOIN_PRICE and MAX_STABLECOIN_PRICE upper and lower bound and the assumption is that the stablecoin will not depeg below 0.95. If the stablecoin does depeg below 0.95, the oracle service is always DOSed.

See for example [this article](#) on CoinMarketCap. In extreme example, USDC shortly depegs to 0.88 before.

Recommendation: Consider add a setter to set the MIN_STABLECOIN_PRICE and MAX_STABLECOIN_PRICE.

SuperEarn: Fixed in PR 8.

Cantina Managed: Fix verified.

3.4.8 `OriginVault.emergencyWithdrawFromAgent` can trigger double-counting errors

Severity: Low Risk

Context: `OriginVault.sol#L1117-L1120`

Description: If the highlighted method is called with `token` parameter set to `OriginVault.asset` and the specified `amount` is accounted for within `SuperEarnMessageAgent.getAssetsInTransit`, such amount of assets will be double-counted permanently within `OriginVault.totalAssets`, leading to incorrect share pricing.

Recommendation: Prevent the method from transferring the vault's asset.

SuperEarn: Fixed in commit `0e16473c`.

Cantina Managed: Fix verified.

3.4.9 `batchFulfillRedemptions` contains unbounded loop in `OriginVault.sol`

Severity: Low Risk

Context: `OriginVault.sol#L554-L572`

Description: The code always attempts to looping over all `redemptionQueue` items and the share holder can fill in the `redemptionQueue` with tiny shares to grow the `redemptionQueue` size infinitely and make the `batchFulfillRedemptions` not-callable.

Recommendation: Consider add a function to fill the individual redeem request and remove the request from the queue.

SuperEarn: Fixed in commit `35f47281`.

Cantina Managed: Fix verified.

3.4.10 `estimatedTotalAssets` should use `previewRedeem` and instead of `previewMint` to estimate total asset

Severity: Low Risk

Context: `StrategyMorphoV1Vault.sol#L379-L390`

Description: `getAmountByShares` calls:

```
function getAmountByShares(uint256 shares) public view virtual override returns
→ (uint256) {
    return metaMorpho.previewMint(shares);
}
```

However, the function tries to evaluate the estimated current total asset. `previewMint(shares)` answers: "how many ASSETS must I pay to mint exactly shares new shares right now?". That's the mint direction (assets → shares), with rounding up. But we want: "how many ASSETS would I receive if I redeem them?".

Recommendation: Call `previewRedeem` instead of `previewMint` when calling `getAmountByShares`.

SuperEarn: Fixed in commit `47ff4635`.

Cantina Managed: Fix verified.

3.4.11 The system heavily relies on unbounded loops

Severity: Low Risk

Context: `OriginVault.sol#L567-L604`, `OriginVault.sol#L823-L851`

Description: Functions within `OriginVault` rely on unbounded loops to accumulate values needed for specific calculations, resulting in many areas in which the system's gas consumption may grow unexpectedly and potentially turn into a full DoS on the affected functions.

Recommendation: The protocol should heavily consider removing any unbounded loop within the system, especially those executed over data sets which may grow as a consequence of end-user interactions. The accumulators which used such loops should rely on values that are accumulated and stored within a contract's storage.

SuperEarn: Fixed in commit [35f47281](#).

Cantina Managed: Fix verified.

3.4.12 Calls to `CooldownVault.recover` dilute value for all share holders

Severity: Low Risk

Context: [CooldownVault.sol#L864-L879](#)

Description: `CooldownVault.recover` may be used by an authorized account to manually mint an amount of shares such that the asset to share exchange rate is set to 1. This will result in all vault depositors losing value they've accrued while providing liquidity to the vault.

Recommendation: Do not allow for manual share price manipulation, as shares appreciating with regards to assets is a common and expected behaviour of ERC4626-compliant vaults.

SuperEarn: Acknowledged.

Cantina Managed: Acknowledged.

3.4.13 Two-step execution pattern in `BaseCooldownStrategy` lacks access control separation and time delay enforcement

Severity: Low Risk

Context: [BaseCooldownStrategy.sol#L389-L443](#)

Description: `BaseCooldownStrategy` implements logic to allow for an authorized party to execute an arbitrary call. This functionality is implemented with 2 methods, `BaseCooldownStrategy.submitExecution` and `BaseCooldownStrategy.acceptExecution`. The way in which these are implemented offer little to no benefit, as:

1. `BaseCooldownStrategy.submitExecution` and `BaseCooldownStrategy.acceptExecution` may be called by the same account with the governance role.
2. There is no minimum time delay between proposal and execution, as is typical, but a deadline is imposed.

Recommendation: The methods should be refactored to either:

1. Require accounts with different roles to call each method.
2. Implement a timelock functionality which forces the methods to be called with a reasonable delay between each other.

SuperEarn: Fixed in [PR 10](#).

Cantina Managed: Fix verified.

3.5 Gas Optimization

3.5.1 Execute allowance check early

Severity: Gas Optimization

Context: [CooldownVaultFactory.sol#L55-L58](#)

Description: The highlighted allowance check within `CooldownVaultFactory.createCooldownVault` should be executed before a new instance of `CooldownVault` is deployed in order to reduce gas in the unlikely case in which such check triggers a revert.

Recommendation: Execute the allowance check before the `CooldownVault` deployment.

SuperEarn: Fixed in commit [77949201](#).

Cantina Managed: Fix verified.

3.5.2 Mathematical calculation can be simplified

Severity: Gas Optimization

Context: [CooldownVault.sol#L865](#)

Description: `CooldownVault#L865` can be simplified by removing `10 ** _decimalsOffset()` given that `_decimalsOffset() == 1`.

Recommendation: Apply the recommended simplifications.

SuperEarn: Fixed in commit [b53b9f06](#).

Cantina Managed: Fix verified.

3.5.3 `SuperEarnAccessControl._isContract` is used only within permissioned methods

Severity: Gas Optimization

Context: [SuperEarnAccessControl.sol#L341-L343](#)

Description: `SuperEarnAccessControl._isContract` is used only within permissioned methods to when writing values to storage. It is thus not strictly necessary to execute such check, given that the permissioned actor is able to verify beforehand whether a given account holds code or not.

Recommendation: Remove the method in order to reduce the gas consumption required for methods using such method internally.

SuperEarn: Fixed in commit [3bec4b2d](#).

Cantina Managed: Fix verified.

3.5.4 Avoid try/catch-ing external calls to revert in the catch clause

Severity: Gas Optimization

Context: [UniversalSwapRouter.sol#L114-L126](#)

Description: The highlighted try/catch block to decode a bytes array provided as a call argument can be entirely avoided by requiring the `data` parameter to be of `SwapConfig[]` type instead of `bytes`.

Recommendation: Change the type of the `data` argument in order to avoid having to make an external call to self and remove the highlighted try/catch block.

SuperEarn: Fixed in commit [d3224384](#).

Cantina Managed: Fix verified.

3.5.5 Unnecessary `Math.min` call

Severity: Gas Optimization

Context: [RemoteVault.sol#L651](#)

Description: `Math.min(availUsdt, neededUsdt)` is superfluous within the highlighted line of code, given that it has already been asserted that `availUsdt >= neededUsdt`.

Recommendation: Remove `Math.min(availUsdt, neededUsdt)` and use `neededUsdt` instead.

SuperEarn: Fixed in commit [918235d3](#).

Cantina Managed: Fix verified.

3.5.6 Token transfers in `SuperEarnMessageAgent` can be simplified into one

Severity: Gas Optimization

Context: `SuperEarnMessageAgent.sol#L293-L296`

Description: The highlighted token transfers should be condensed into one, in order to avoid executing two separate external calls.

Recommendation: Modify the highlighted transfers to

```
IERC20(token).safeTransferFrom(address(adapter), targetVault, amount);
```

SuperEarn: Fixed in [PR 5](#).

Cantina Managed: Fix verified.

3.6 Informational

3.6.1 Incorrect comment

Severity: Informational

Context: `UniversalSwapRouter.sol#L186`

Description: The highlighted comments present errors that should be fixed to ensure the documentation's correctness:

- Within `UniversalSwapRouter::L186` : "Curve" should be corrected to "Uniswap".

Recommendation: Apply the suggestions provided in the section above.

SuperEarn: Fixed in commit [57684761](#).

Cantina Managed: Fix verified.

3.6.2 `UniversalSwapRouter` uses two libraries for safe `ERC20` interactions

Severity: Informational

Context: `UniversalSwapRouter.sol#L6-L10`

Description: `SafeERC20` and `TransferHelper` serve the same purpose so there is no need to use both of them within the same contract.

Recommendation: Select and use only one of the two libraries.

SuperEarn: Fixed in commit [8936ee34](#).

Cantina Managed: Fix verified.

3.6.3 `CrosschainAdapter.sendAssets` will incorrectly account for messages sent to unregistered networks

Severity: Informational

Context: `CrosschainAdapter.sol#L258-L278`

Description: `CrosschainAdapter.sendAssets` may be called by a `SuperEarnMessageAgent` instance of by an authorized vault to trigger a crosschain asset transfer.

After validating the caller and pulling the requested funds, the highlighted method will initiate the crosschain message accounting with a call to `SuperEarnMessageAgent.notifyBridgeInitiated`. Once the external call has been executed, only if `peerAdapters[destinationChainId] != address(0)` the method will actually trigger the entire crosschain message dispatch via the protocol's custom Runespear messaging format and CCIP integration.

While rather unlikely, if the caller were to provide a `destinationId` which hasn't been registered yet in the contract's storage, the `SuperEarnMessageAgent.notifyBridgeInitiated` method is triggered without a crosschain message actually being initiated.

Recommendation: Within the highlighted method, revert if

```
peerAdapters[destinationChainId] == address(0)
```

to disallow this edge case from being processed.

SuperEarn: Fixed in PR 5.

Cantina Managed: Fix verified.

3.6.4 Incorrect parameter in external call within `CrosschainAdapter.sendAssets`

Severity: Informational

Context: `CrosschainAdapter.sol#L258`

Description: The highlighted method calls `agent.notifyBridgeInitiated(callingVault, ...)`. The method's documentation states the first parameter to be the vault that must be notified of the bridge. Given that `callingVault` may be assigned either `CrosschainAdapter.vault` or `CrosschainAdapter.agent`, the provided parameter will be incorrect in the case in which the method is called by the latter.

Recommendation: Always provide `CrosschainAdapter.vault` to the call.

SuperEarn: Fixed in PR 5.

Cantina Managed: Fix verified.

3.6.5 Empty method implements access control

Severity: Informational

Context: `SuperEarnMessageAgent.sol#L256-L271`

Description: `SuperEarnMessageAgent.notifyBridgeInitiated` is a no-op method with an empty body, except for validating that `msg.sender == address(adapter)`.

Recommendation: Remove the method if it is not necessary or remove the access control.

SuperEarn: Fixed in PR 5.

Cantina Managed: Fix verified.

3.6.6 `SuperEarnAccessControl` modifiers should use the contract's getters

Severity: Informational

Context: `SuperEarnAccessControl.sol#L226-L307`, `SuperEarnAccessControl.sol#L354-L401`

Description: The highlighted modifiers duplicate code instead of utilizing the public view methods it defines.

Recommendation: In order to improve the code's readability and maintainability, consider using the public getter methods.

SuperEarn: Fixed in commits 012bfd05 and 3569072a.

Cantina Managed: Fix verified.

3.6.7 Variable assignment can be simplified

Severity: Informational

Context: `CrosschainAdapter.sol#L255`

Description: `CrosschainAdapter.sendAssets` may only be called by `CrosschainAdapter.agent` or `CrosschainAdapter.vault`. The assignment of `tokenSource` at `CrosschainAdapter#L255` can thus be simplified.

Recommendation: Modify the highlighted line of code to assign `tokenSource = msg.sender`.

SuperEarn: Fixed in PR 5.

Cantina Managed: Fix verified.

3.6.8 Incorrect parameter name in interface's event

Severity: Informational

Context: `IERC7540.sol#L96-L98`

Description: Within `IERC7540.sol`, the declaration of `RedeemRequest` event incorrectly names the last parameter to "assets" instead of "shares".

Recommendation: Rename the mentioned parameter.

SuperEarn: Fixed in commit `5e1072a3`.

Cantina Managed: Fix verified.

3.6.9 Initial infinite approval from `OriginVault` to `SuperEarnMessageAgent` is overwritten

Severity: Informational

Context: `OriginVault.sol#L216-L240`, `OriginVault.sol#L478-L484`

Description: When `OriginVault.setAgent` is called, an infinite allowance of `asset` is given to the newly written agent. This approval is overwritten within `OriginVault.depositToRemote`.

Recommendation: Remove the unnecessary asset approval within `OriginVault.depositToRemote`.

SuperEarn: Fixed in commit `e026f0c4`.

Cantina Managed: Fix verified.

3.6.10 Unreachable code

Severity: Informational

Context: `UniversalSwapRouter.sol#L370-L377`, `UniversalSwapRouter.sol#L429-L430`

Description: `UniversalSwapRouter#L429-430` is unreachable code given that `UniversalSwapRouter._swapExactInputSingleUniSwapV4` always creates a `CallbackData` struct with `callbackData.settleUsingBurn = false`.

Recommendation: Remove the dead code.

SuperEarn: Fixed in commits `cb3aa411` and `86f3e7c4` (removal of `settleUsingBurn`, `tokenIn` and `tokenOut` parameters from the struct).

Cantina Managed: Fix verified.

3.6.11 `HealthCheck.setGovernance` is missing access control

Severity: Informational

Context: `HealthCheck.sol#L28-L31`

Description: `HealthCheck.setGovernance` isn't applied the `onlyGovernance` modifier.

Recommendation: Even though it is implemented by `CommonHealthCheck.setGovernance`, which implements proper access control for the highlighted method, the protocol should consider applying the modifier to keep the method consistent with all other methods.

SuperEarn: Fixed in [PR 10](#).

Cantina Managed: Fix verified.

3.6.12 Unnecessary ternary operator

Severity: Informational

Context: `StrategyERC7540Vault.sol#L592`

Description: The ternary operator at the highlighted line of code isn't necessary, given that it has already been asserted that `_amountNeeded > pendingInvested`.

Recommendation: Remove the ternary operator and use `_amountNeeded - pendingInvested`.

SuperEarn: Fixed in commit `f16da2bc`.

Cantina Managed: Fix verified.

3.6.13 Bridge accounting assumes single asset type and fails to track per-token balances

Severity: Informational

Context: `RemoteVault.sol#L734-L758`

Description: The `IRunespearAgent` + `IBridgeAccountant` design comment says:

```
// CRITICAL NOTE: now this assumes a single kind of assets (e.g. USDC)
```

The current runespear and cross-chain adapter does not track the bridge amount per token. The token bridged out can be USDT or the asset.

```
address token = tokenType == SuperEarnV2Protocol.AssetType.USDT ? usdt : asset();
```

Recommendation: Support multiple token requires changes to the cross-chain protocol, so in the current codebase, the protocol should swap to target token before bridging.

SuperEarn: Fixed in commit `96e1690e`.

Cantina Managed: Fix verified.

3.6.14 `CooldownVault` isn't ERC4626 compliant

Severity: Informational

Context: `CooldownVault.sol#L28`

Description: The contract isn't fully IERC4626 compliant given that it:

1. May not return funds upon calls to `CooldownVault.withdraw` and `CooldownVault.redeem`.
2. Changes the semantics of the values returned by such methods: they're now request ids instead of share or asset amounts.

Recommendation: The protocol should consider implementing ERC7540, as it is formally more correct for this contract.

SuperEarn: Acknowledged. `cooldownVault` is not actual ERC4626 now. it just uses interface of `IERC4626`, and we use `ERC20WrapperUpgradable`.

Cantina Managed: Acknowledged.